PRAKTIKUM APLIKASI WEB

Membangun Chatbot Tematik dengan Flask

Teori dari dasar: konsep chatbot, web dan HTTP, Python dan Flask, pemrosesan teks, REST API, data, pengujian, dan evaluasi

AGENDA

Delapan bagian, dari konsep sampai aplikasi jadi

01

Konsep chatbot

Jenis chatbot, istilah, desain tema

02

Dasar web

Client-server, URL, HTTP, JSON, HTML/CSS/JS

03

Python dan Flask

venv, route, template, form

04

Mesin chatbot

Praproses, skor kemiripan, ambang

05

API dan antarmuka

REST API, fetch(), flowchart

06

Aplikasi final

Sesi, SQLite, admin, keamanan

07

Pengujian dan evaluasi

pytest, akurasi, keterbatasan

08

Hasil dan tugas

Tampilan, deployment, rubrik

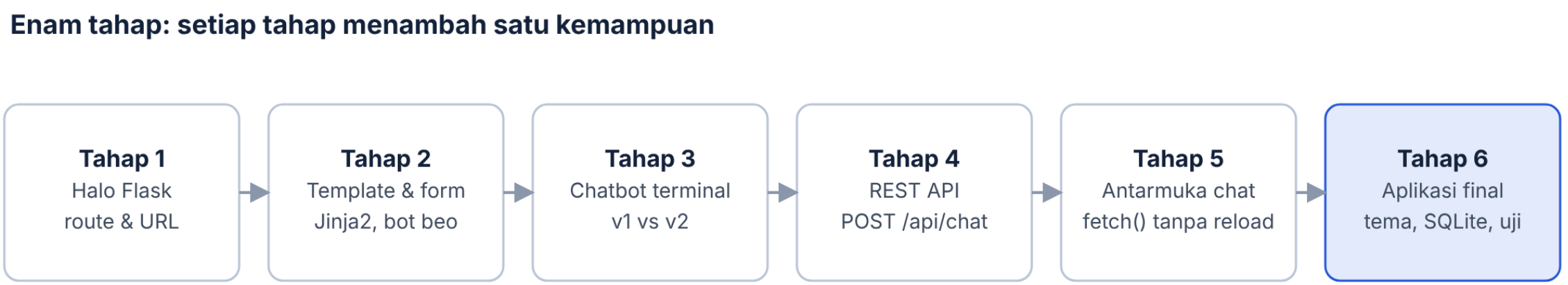
CAPAIAN PEMBELAJARAN

Setelah praktikum, mahasiswa mampu

1. Menjelaskan cara kerja chatbot berbasis aturan dan bedanya dengan chatbot ML/LLM
2. Merancang basis pengetahuan tematik: intent, pola, respons, fallback
3. Menjelaskan alur HTTP request/response, URL, metode, kode status, dan JSON
4. Membangun aplikasi Flask dengan route, template Jinja2, dan form
5. Membuat mesin chatbot: praproses teks, skor kemiripan, ambang batas
6. Membuat REST API dan antarmuka chat dengan JavaScript fetch()
7. Menyimpan riwayat di SQLite, menguji dengan pytest, dan mengukur akurasi
8. Menerapkan pengaman dasar: XSS, SQL injection, validasi input, rahasia

PETA BELAJAR

Enam tahap, setiap tahap menambah satu kemampuan

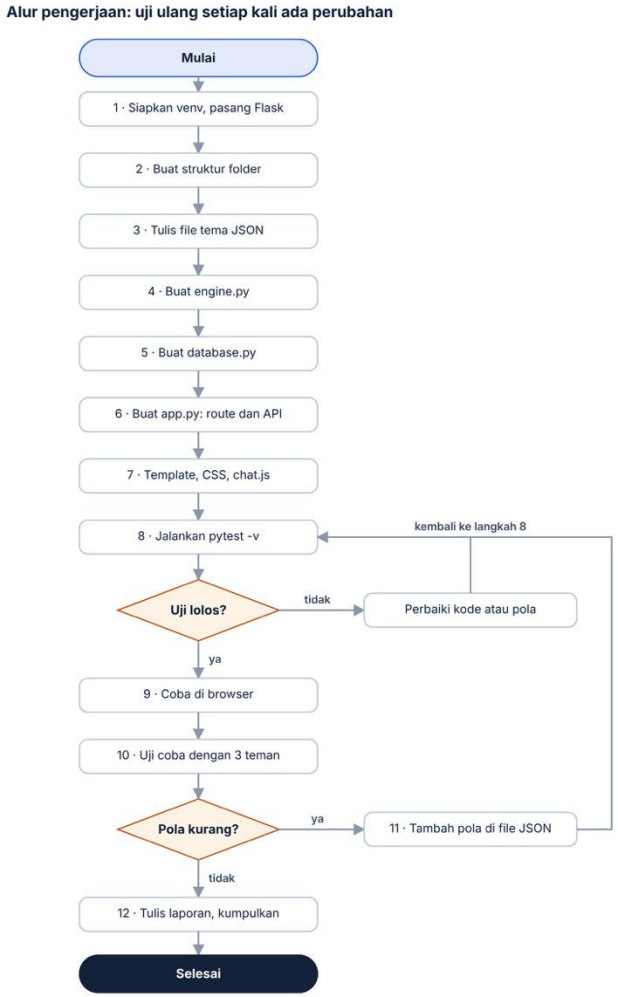


Setiap folder di tahapan/ bisa dijalankan sendiri dengan python app.py

Tahap	Folder	Konsep baru
1	tahap-01-halo-flask	Route, variabel URL, query string, jsonify
2	tahap-02-template-dan-form	Template Jinja2, form POST, bot beo
3	tahap-03-chatbot-terminal	Mesin chatbot v1 vs v2, tanpa web
4–5	tahap-04-api-json · tahap-05-antarmuka-chat	REST API, fetch(), tanpa reload
6	aplikasi-final	Multi tema, SQLite, admin, uji, evaluasi

PETA BELAJAR

Alur pengerjaan: uji ulang setiap ada perubahan



Kerjakan berurutan

Mulai dari lingkungan dan data tema, lalu mesin, database, web, dan tampilan. Setiap lapisan bergantung pada lapisan sebelumnya.

Uji sesering mungkin

Jalankan pytest -v setiap selesai mengubah file. Kesalahan kecil lebih mudah dicari daripada kesalahan yang menumpuk.

Perbaiki dari data nyata

Pertanyaan teman yang tidak dikenali adalah sumber pola terbaik. Lihat di halaman admin, tambahkan ke JSON.

01

Konsep Chatbot

Sebelum menulis kode, pahami apa itu chatbot, jenis-jenisnya, dan apa yang membuat chatbot tematik efektif.

DEFINISI

Chatbot: program yang bercakap dengan manusia lewat teks

Chatbot menerima pesan berbahasa alami, menebak maksudnya, lalu memberi jawaban yang sesuai. Sebagian besar chatbot layanan bertugas menjawab pertanyaan yang sering diajukan (FAQ).



Layanan kampus

Jadwal KRS, cara bayar UKT, jam perpustakaan, beasiswa



Toko daring

Status pesanan, ongkos kirim, cara retur barang



Pariwisata

Destinasi, transportasi, waktu terbaik berkunjung



Layanan publik

Jam layanan, syarat dokumen, alur pendaftaran

Kesamaan keempatnya: pertanyaan berulang dalam satu domain, jawabannya sudah diketahui. Inilah lahan ideal chatbot tematik.

Tematik berarti domainnya sengaja dibatasi

Fokus

Hanya menjawab satu domain, misalnya layanan akademik. Pertanyaan lain ditolak dengan sopan lewat fallback.

Akurat

Setiap jawaban ditulis dan diperiksa manusia, sehingga bot tidak mengarang informasi.

Terukur

Domain terbatas berarti daftar pertanyaan bisa dibuat, sehingga akurasinya bisa dihitung.

Tiga tema di praktikum ini

Satu kode Python yang sama dipakai untuk ketiganya; yang berbeda hanya file JSON.

Tema	Nama bot	Contoh pertanyaan	Jumlah intent
Asisten Akademik Kampus	SiAkad	Kapan pengisian KRS? Cara bayar UKT?	12
Pemandu Kuliner Nusantara	Mbok Rasa	Apa itu gudeg? Resep nasi goreng?	10
Pemandu Wisata Yogyakarta	JogjaBot	Ada apa di Malioboro? Naik apa keliling Jogja?	10

JENIS CHATBOT

Tiga pendekatan chatbot, kita memakai yang berbasis aturan

Pendekatan	Cara kerja	Kelebihan	Kekurangan
Berbasis aturan (dipakai di sini)	Mencocokkan pesan ke pola yang ditulis manual	Transparan, ringan, tanpa data latih, jawaban pasti benar	Hanya tahu apa yang ditulis; lemah pada sinonim
Machine learning (klasifikasi intent)	Model belajar dari ratusan contoh kalimat berlabel	Lebih luwes terhadap variasi kalimat	Butuh data latih dan proses pelatihan
LLM (model bahasa besar)	Model besar menyusun jawaban sendiri	Sangat luwes, percakapan alami	Butuh API dan biaya; bisa mengarang jawaban

Rule-based dipilih karena setiap jawaban bisa dijelaskan: mahasiswa melihat persis pola mana yang cocok dan berapa skornya.

ANATOMI CHATBOT

Chatbot punya tiga tugas: memahami, memutuskan, menjawab

1 · Memahami (NLU)

Mengubah kalimat menjadi maksud (intent).

`engine.py: preprocess() + similarity()`

2 · Memutuskan

Memilih intent terbaik atau menolak bila tidak yakin.

`classify() + ambang 0,45`

3 · Menjawab (NLG)

Menyusun kalimat jawaban untuk pengguna.

`reply(): respons acak + placeholder`

Chatbot ML dan LLM mengganti satu atau lebih kotak ini dengan model; alurnya tetap sama.

ISTILAH KUNCI

Enam istilah yang dipakai sampai akhir

Tema

Domain pengetahuan bot. Satu tema = satu file JSON.

Contoh: kampus, kuliner, wisata

Intent

Maksud pengguna yang ingin dikenali.

Contoh: salam, krs, ukt

Pattern (pola)

Contoh kalimat yang mewakili satu intent.

Contoh: “cara bayar ukt”

Response

Daftar jawaban; satu dipilih acak.

Contoh: “UKT dibayar melalui…”

Fallback

Jawaban saat tidak ada intent yang cukup mirip.

Contoh: “Maaf, saya belum…”

Threshold (ambang)

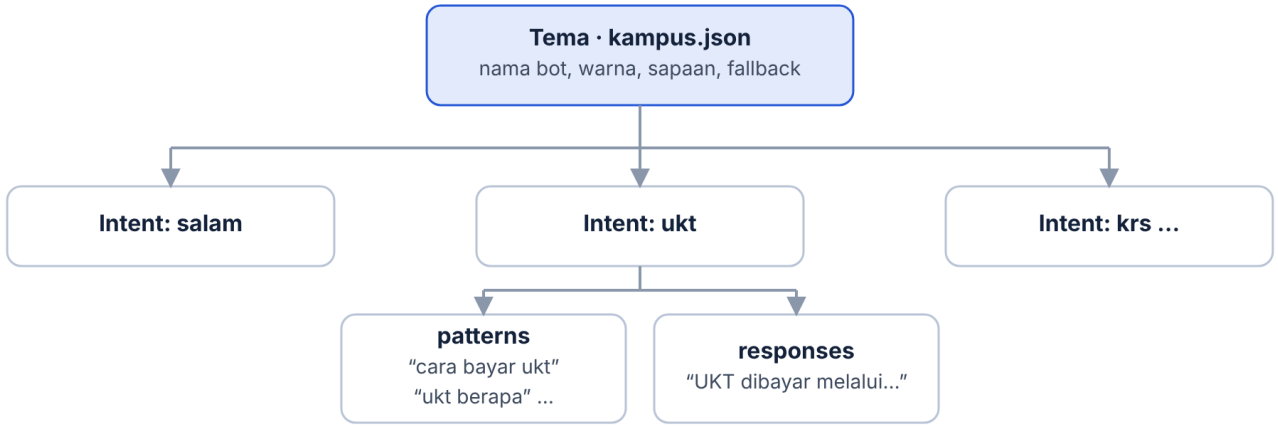
Skor minimum agar sebuah intent dianggap cocok.

Contoh: 0,45

BASIS PENGETAHUAN

Satu file tema berisi banyak intent

Isi satu file tema: banyak intent, setiap intent punya pola dan respons



- Satu intent = satu topik pertanyaan
- 4–8 pola per intent, dengan kata yang benar-benar dipakai orang
- Minimal 2 respons agar tidak monoton
- Fallback menyebut topik yang bisa dijawab

themes/kampus.json

```
{
 "id": "kampus",
 "bot_name": "SiAkad",
 "fallback": ["Maaf, saya belum ..."],
 "intents": [
  {"tag": "ukt",
   "patterns": ["ukt",
     "cara bayar ukt",
     "batas bayar ukt"],
   "responses": [
     "UKT dibayar melalui ..."]}
 ]
}
```

Bot yang baik punya persona dan menolak dengan membantu



Dianjurkan

- Nama dan gaya bahasa konsisten (SiAkad sopan, Mbok Rasa hangat)
- Sapaan awal menyebut topik yang dikuasai
- Fallback mengarahkan: “Coba tanyakan tentang KRS, UKT…”
- Tombol saran untuk pertanyaan populer
- Jawaban singkat, langsung, satu topik



Dihindari

- Fallback buntu: “Saya tidak mengerti.”
- Menjawab hal di luar tema dengan asal
- Jawaban panjang berisi banyak topik sekaligus
- Mengarang data (tanggal, angka) yang belum pasti
- Gaya bahasa berganti-ganti antarjawaban

Lima langkah merancang tema sebelum menulis JSON

1

Pilih domain

Sempit dan jelas, misalnya layanan perpustakaan.

2

Kumpulkan pertanyaan

Tanya 5–10 calon pengguna, catat kalimat aslinya.

3

Kelompokkan jadi intent

Pertanyaan dengan maksud sama = satu intent.

4

Tulis pola dan respons

Pola dari kalimat asli; respons singkat dan benar.

5

Uji dan perbaiki

Ukur akurasi, tambah pola untuk yang gagal.

Contoh langkah 3: “kapan KRS dibuka”, “jadwal isi KRS”, “krs online kapan” → satu intent krs.

02

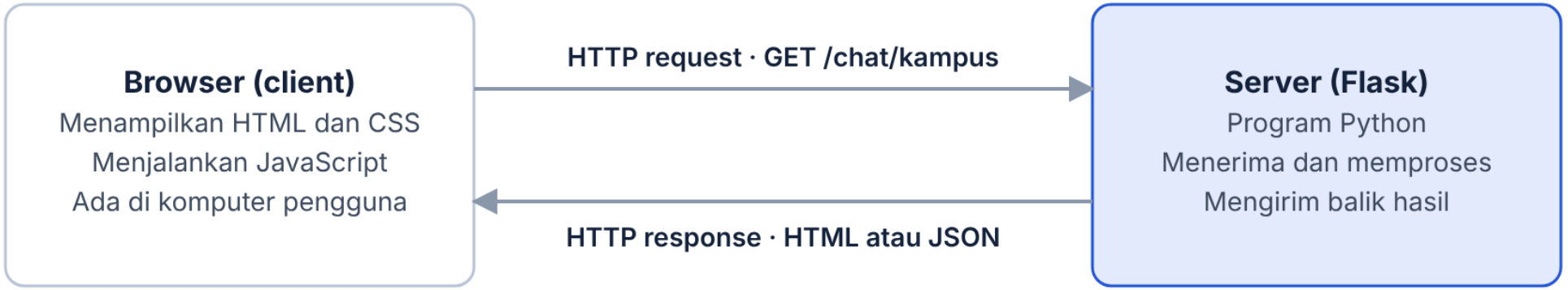
Dasar Aplikasi Web

Chatbot kita hidup di web. Pahami dulu bagaimana browser dan server saling berkirim pesan.

CLIENT-SERVER

Browser meminta, server menjawab

Setiap interaksi web = satu request dan satu response



Client

Pihak yang meminta: halaman chat di browser.

Server

Pihak yang menjawab: app.py yang dijalankan Flask.

HTTP

Aturan format pesan: setiap request dibalas satu response.

Membaca alamat: setiap bagian URL punya arti

http://	127.0.0.1	:5000	/cari	?q=krs
Protokol	Host (komputer sendiri)	Port	Path → route Flask	Query string

URL	Fungsi Flask yang dipanggil	Tahap
/	index()	1
/halo/Budi	halo(nama="Budi")	1
/cari?q=krs	cari() membaca request.args["q"]	1
/chat/kampus	chat_page(theme_id="kampus")	6

Anatomi satu request dan response

Request — dikirim browser

```
POST /api/chat HTTP/1.1
Host: 127.0.0.1:5000
Content-Type: application/json

{"message": "cara bayar ukt",
 "theme": "kampus"}
```

Response — dikirim Flask

```
HTTP/1.1 200 OK
Content-Type: application/json

{"intent": "ukt",
 "reply": "UKT dibayar melalui ...",
 "score": 1.0}
```

Baris pertama

Request: metode + path. Response: kode status (200 = berhasil).

Header

Keterangan tambahan, misalnya Content-Type: isi body berformat JSON.

Body

Data yang dikirim, dipisah dari header oleh satu baris kosong.

Metode dan kode status yang dipakai aplikasi

Metode: apa yang diminta

Metode	Arti	Contoh
GET	Ambil data	/, /chat/kampus, /api/history
POST	Kirim data baru	/api/chat, form bot beo
DELETE	Hapus data	/api/history/kampus

Kode status: bagaimana hasilnya

Kode	Arti	Kapan muncul
200	OK	Berhasil
400	Bad Request	Pesan kosong, tema salah, > 500 karakter
403	Forbidden	Kunci admin salah
404	Not Found	Tema tidak ada
500	Server Error	Ada galat di kode Python

Angka depan menunjukkan kelompok: 2xx berhasil, 4xx kesalahan dari pihak client, 5xx kesalahan di server.

JSON: bahasa bersama Python dan JavaScript

Python	JSON	JavaScript
dict {"a": 1}	object {"a": 1}	Object
list [1, 2]	array [1, 2]	Array
str "halo"	string "halo"	String
int / float	number	Number
True / False	true / false	true / false
None	null	null

```
 # Python (Flask) mengirim
return jsonify({"intent": "ukt",
                "score": 1.0})

 // JavaScript menerima
const data = await res.json();
console.log(data.intent);  // "ukt"
```

JSON adalah teks biasa, jadi bisa dikirim lewat HTTP dan dibaca bahasa apa pun. File tema chatbot juga ditulis dalam JSON.

FRONT-END

HTML, CSS, dan JavaScript punya peran berbeda



HTML · struktur

Menentukan isi halaman: judul, area pesan, kotak input, tombol. File: templates/*.html



CSS · tampilan

Warna, ukuran, tata letak, gelembung chat, tampilan ponsel. File: static/css/style.css



JavaScript · perilaku

Menangkap klik, mengirim pesan dengan fetch(), menambah gelembung jawaban. File: static/js/chat.js

```
<form id="chat-form">                          <!-- HTML -->
  <input id="chat-input">  <button>Kirim</button>
</form>
.msg-user .bubble { background: var(--primary); }  /* CSS */
form.addEventListener("submit", kirim);              // JS
```

JavaScript mengubah halaman lewat DOM dan event

```
const input = document.getElementById("chat-input");
const form = document.getElementById("chat-form");

form.addEventListener("submit", (e) => {
  e.preventDefault();          // jangan muat ulang halaman
  const bubble = document.createElement("div");
  bubble.textContent = input.value;   // aman dari XSS
  document.getElementById("messages").appendChild(bubble);
});
```

Fungsi / properti	Kegunaan
getElementById()	Mengambil satu elemen berdasarkan id
addEventListener()	Menjalankan fungsi saat event terjadi
preventDefault()	Membatalkan perilaku bawaan (reload)
createElement() + appendChild()	Membuat dan menambah elemen baru
textContent	Mengisi teks tanpa menafsirkan HTML

03

Python dan Flask

Bekal Python yang dipakai, lingkungan kerja, lalu Flask: route, template, dan form.

Konsep Python yang muncul di kode chatbot

Konsep	Contoh di kode chatbot	Dipakai untuk
dict dan list	theme["intents"], intent["patterns"]	Menyimpan isi file tema
Fungsi	def preprocess(teks): ... return tokens	Memecah logika menjadi bagian kecil
Class dan objek	bot = ChatbotEngine(theme)	Satu mesin per tema
Set	set(a) & set(b)	Menghitung kata yang sama (Jaccard)
List comprehension	[pola for i in intents for pola in i["patterns"]]	Mengumpulkan semua pola
f-string	f"Tema '{theme_id}' tidak ditemukan"	Menyusun teks pesan
Dekorator	@app.route("/")	Mendaftarkan fungsi ke URL

Virtual environment memisahkan pustaka setiap proyek

```
python -m venv venv                # buat (sekali saja)
venv\Scripts\activate              # aktifkan di Windows
source venv/bin/activate           # aktifkan di macOS/Linux
pip install -r requirements.txt    # Flask + pytest
python app.py                      # jalankan aplikasi
deactivate                         # selesai
```

Tanda berhasil

Awal baris terminal berubah menjadi (venv).

requirements.txt

Daftar pustaka proyek; cukup Flask dan pytest.

- Aktifkan venv setiap kali membuka terminal baru
- Jangan unggah folder venv/ ke repositori (.gitignore)
- PowerShell menolak? Jalankan sekali: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

Flask: framework web kecil dengan tiga konsep inti

1 · Route

Memetakan URL ke fungsi Python.

2 · Template

HTML dengan “lubang” yang diisi data oleh Jinja2.

3 · Request & response

request membawa data dari browser; fungsi mengembalikan HTML atau JSON.

Siklus satu request di Flask



TAHAP 1

Program pertama: route, variabel URL, query string

tahapan/tahap-01-halo-flask/app.py

```
from flask import Flask, jsonify, request

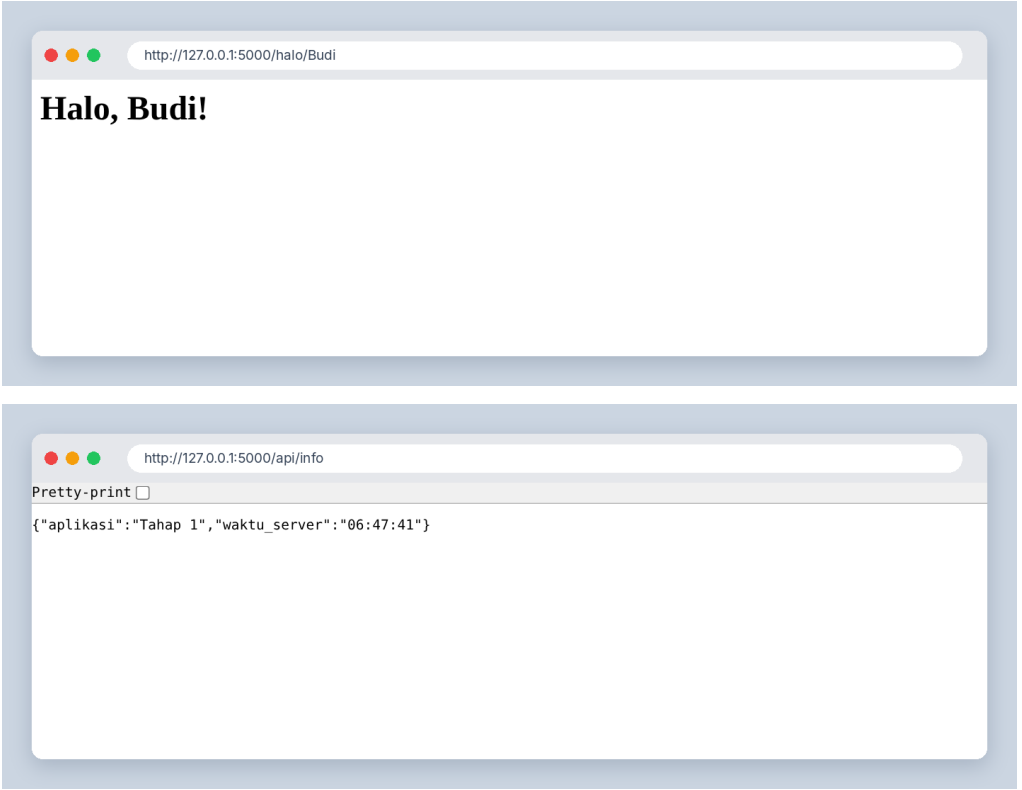
app = Flask(__name__)

@app.route("/")
def index():
    return "<h1>Halo, Flask!</h1>"

@app.route("/halo/<nama>")        # variabel URL
def halo(nama):
    return f"<h1>Halo, {nama}!</h1>"

@app.route("/cari")               # query string
def cari():
    q = request.args.get("q", "")
    if not q:
        return "Tambahkan ?q=kata", 400
    return f"Anda mencari: <b>{q}</b>"

if __name__ == "__main__":
    app.run(debug=True)
```



Atas: /halo/Budi · Bawah: /api/info mengembalikan JSON

debug=True: server memuat ulang otomatis saat file disimpan. Jangan dipakai di server sungguhan.

Template Jinja2: HTML yang diisi data dari Python

Sintaks	Fungsi
{{ … }}	Menampilkan nilai, otomatis di-escape agar aman
{% for %}	Perulangan, misalnya satu kartu per tema
{% if %}	Percabangan di dalam HTML
{% extends %} {% block %}	Mewarisi kerangka base.html
| length, | tojson	Filter: ubah atau hitung nilai

tahap-02 · templates/index.html

```
{% extends "base.html" %}
{% block content %}
<h1>Daftar tema</h1>
<p>Jumlah tema: {{ tema | length }}</p>
<ul>
  {% for t in tema %}
  <li>{{ t.avatar }} <b>{{ t.nama }}</b></li>
  {% endfor %}
</ul>
{% endblock %}
```

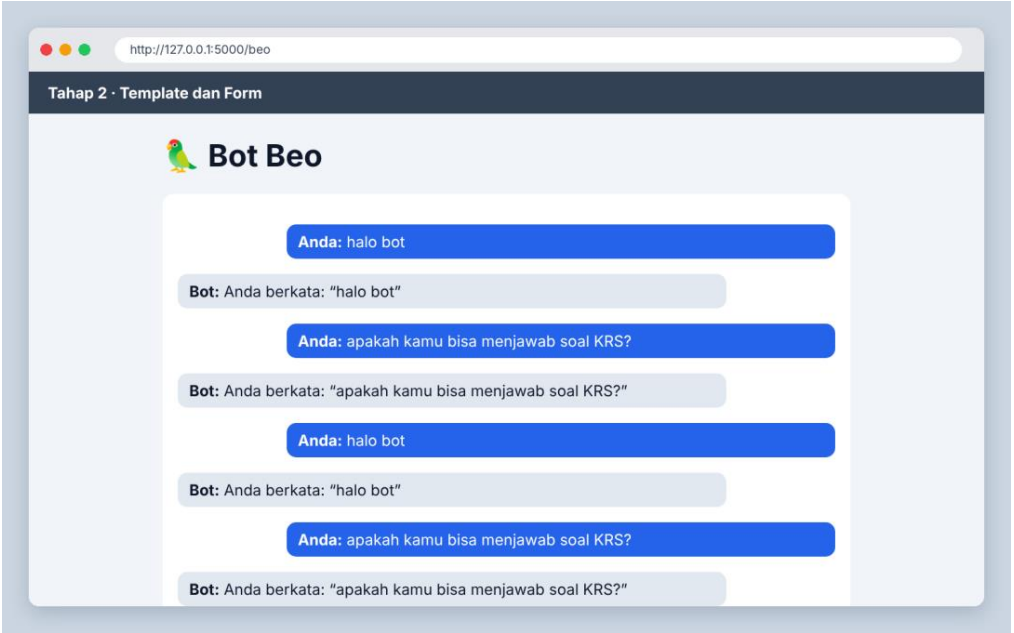
```
return render_template("index.html", tema=TEMA)
```

Data dikirim dari fungsi view ke template lewat argumen render_template()

TAHAP 2

Form POST dan “bot beo”: interaksi pertama dengan pengguna

```
@app.route("/beo", methods=["GET", "POST"])
def beo():
    if request.method == "POST":
        pesan = request.form.get("pesan", "").strip()
        if pesan:
            balasan = f"Anda berkata: “{pesan}”"
            percakapan.append(("Anda", pesan))
            percakapan.append(("Bot", balasan))
    return render_template("beo.html",
                           percakapan=percakapan)
```



Yang sudah bisa

Menerima input pengguna dan membalasnya lewat server.

Kekurangan

Setiap kirim pesan, seluruh halaman dimuat ulang. Diperbaiki di Tahap 5.

STRUKTUR PROYEK

Struktur aplikasi final: setiap folder satu tugas

```
aplikasi-final/
├── app.py              # route halaman + REST API + admin
├── cli.py              # chat di terminal
├── evaluate.py         # ukur akurasi
├── chatbot/
│   ├── engine.py       # praproses, skor, validasi tema
│   └── database.py     # riwayat & statistik SQLite
├── themes/             # 1 file JSON = 1 tema
├── evaluasi/           # data uji CSV per tema
├── templates/          # base, index, chat, admin, 404
├── static/             # css/style.css, js/chat.js
└── tests/              # 34 uji pytest
```

Konvensi Flask

HTML wajib di templates/, CSS dan JS di static/.

Logika terpisah

chatbot/ tidak mengimpor Flask, jadi mudah diuji.

Data terpisah

Ganti tema cukup dengan mengubah file JSON.

04

Mesin Chatbot

Inti praktikum: bagaimana kalimat diubah menjadi intent, diberi skor, lalu dijawab.

MASALAHNYA

Satu maksud bisa ditulis dengan banyak cara

Kalimat pengguna	Skor tertinggi ke intent ukt
cara bayar ukt	1,00
Gimana cara bayar UKT?	0,82
batas pembayaran ukt kapan	0,71
ukt bayarnya gimana	0,51
uang kuliah tunggal dibayar kemana	0,32 ✗

Skor dihitung dengan engine.py aplikasi final; ambang 0,45

Variasi yang harus ditangani

Huruf besar-kecil, tanda baca, kata tidak baku (gimana, gmn), salah ketik, urutan kata, sinonim.

Batas pendekatan ini

Sinonim (“uang kuliah tunggal”) gagal dikenali kecuali ditambahkan sebagai pola.

GAMBARAN BESAR

Lima langkah dari pesan ke jawaban




```
tokens = preprocess(pesan)                  # langkah 2
skor = similarity(tokens, pola_tokens)     # langkah 3, untuk setiap pola
if skor_terbaik < 0.45: jawab_fallback()   # langkah 5
```

PRAPROSES

Praproses menyeragamkan kalimat sebelum dibandingkan

Langkah	Istilah NLP	Hasil
Kalimat asli	—	Gimana cara bayar UKT yang benar, kak?
1 · Huruf kecil	case folding	gimana cara bayar ukt yang benar, kak?
2 · Buang tanda baca	cleaning	gimana cara bayar ukt yang benar kak
3 · Normalisasi	normalization	bagaimana cara bayar ukt yang benar (kak dihapus)
4 · Pecah dan buang kata henti	tokenizing + stopword removal	[bagaimana, cara, bayar, ukt, benar]

```
def preprocess(teks):
    teks = re.sub(r"[^a-z0-9\s]", " ", teks.lower())
    tokens = []
    for kata in teks.split():
        for k in NORMALISASI.get(kata, kata).split():
            if k and k not in STOPWORDS:
                tokens.append(k)
    return tokens
```

Kamus sendiri

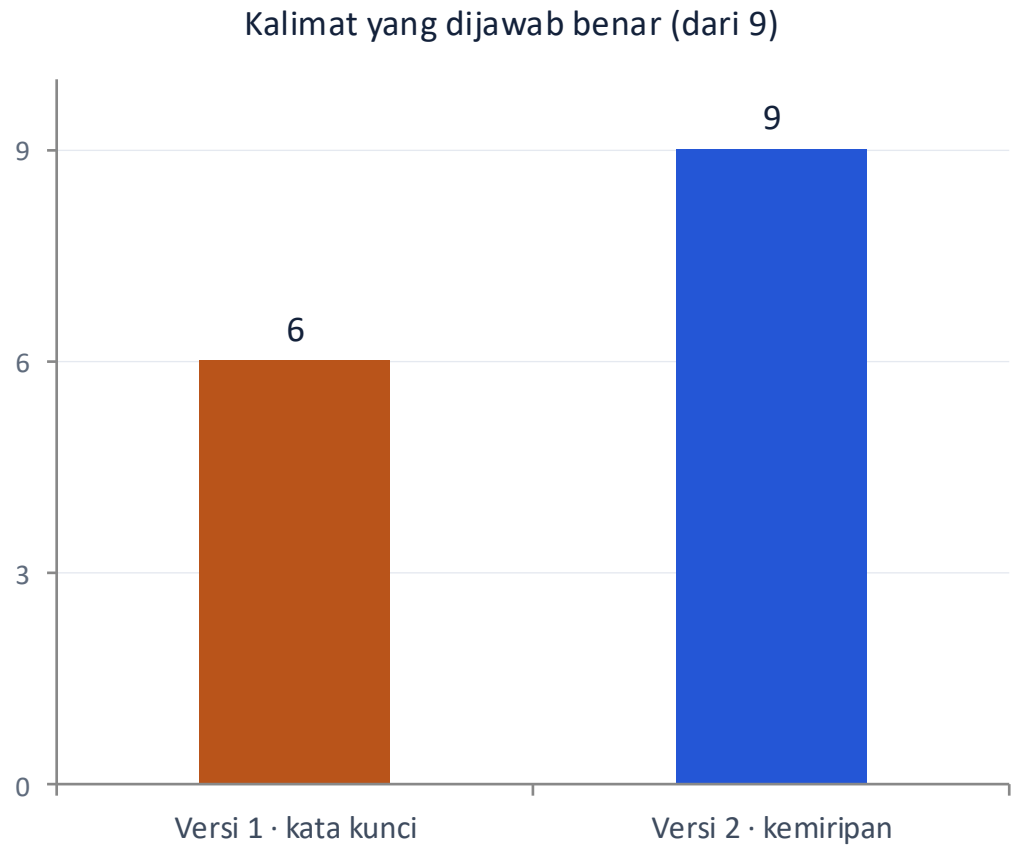
NORMALISASI: gmn → bagaimana, kak → dihapus. STOPWORDS: yang, di, apa, mau… Tambahkan sesuai tema Anda.

Versi 1: pencocokan kata kunci persis, cepat tapi rapuh

engine_v1_kata_kunci.py (inti)

```
for intent in theme["intents"]:
    for pola in intent["patterns"]:
        if pola.lower() in pesan.lower():
            return intent["tag"]  # pertama menang
return None
```

Kalimat	Versi 1	Versi 2
jadwl kuliah	✗ gagal	✓ jadwal_kuliah
perpus buka jam berapa	✗ gagal	✓ perpustakaan
halo, kapan pengisian krs?	✗ salam	✓ krs
syarat beasiswa apa aja	✓ beasiswa	✓ beasiswa



Sumber: tahapan/tahap-03-chatbot-terminal/bandingkan.py

Skor kemiripan = gabungan tiga ukuran

$$\text{skor} = 0{,}4 \times \text{Jaccard} + 0{,}3 \times \text{Fuzzy} + 0{,}3 \times \text{Cakupan}$$

Jaccard · 40%

Kata yang sama ÷ semua kata gabungan.
Mengukur kesamaan kosakata.

Fuzzy · 30%

Rasio kemiripan huruf (difflib.SequenceMatcher).
Tahan salah ketik: “jadwl” ≈ “jadwal”.

Cakupan · 30%

Kata pola yang muncul di pesan ÷ jumlah kata pola.
Menghargai pesan panjang yang memuat seluruh pola.

Nilai 1,00 = pesan dan pola sama persis setelah praproses. Bobot 0,4 / 0,3 / 0,3 bisa diubah sebagai eksperimen.

VERSI 2 · CONTOH SALAH KETIK

Menghitung skor “jadwl kuliah” terhadap pola “jadwal kuliah”

Ukuran	Perhitungan	Nilai
Token pesan	[jadwl, kuliah]	—
Token pola	[jadwal, kuliah]	—
Jaccard	1 kata sama (kuliah) ÷3 kata gabungan	0,33
Fuzzy	kemiripan huruf “jadwl kuliah” vs “jadwal kuliah”	0,96
Cakupan	1 dari 2 kata pola muncul	0,50
Skor	0,4×0,33 + 0,3×0,96 + 0,3×0,50	0,57

0,57

≥ 0,45 → intent jadwal_kuliah dikenali meski salah ketik

0,96

kontribusi terbesar datang dari fuzzy (kemiripan huruf)

VERSI 2 · CONTOH LENGKAP

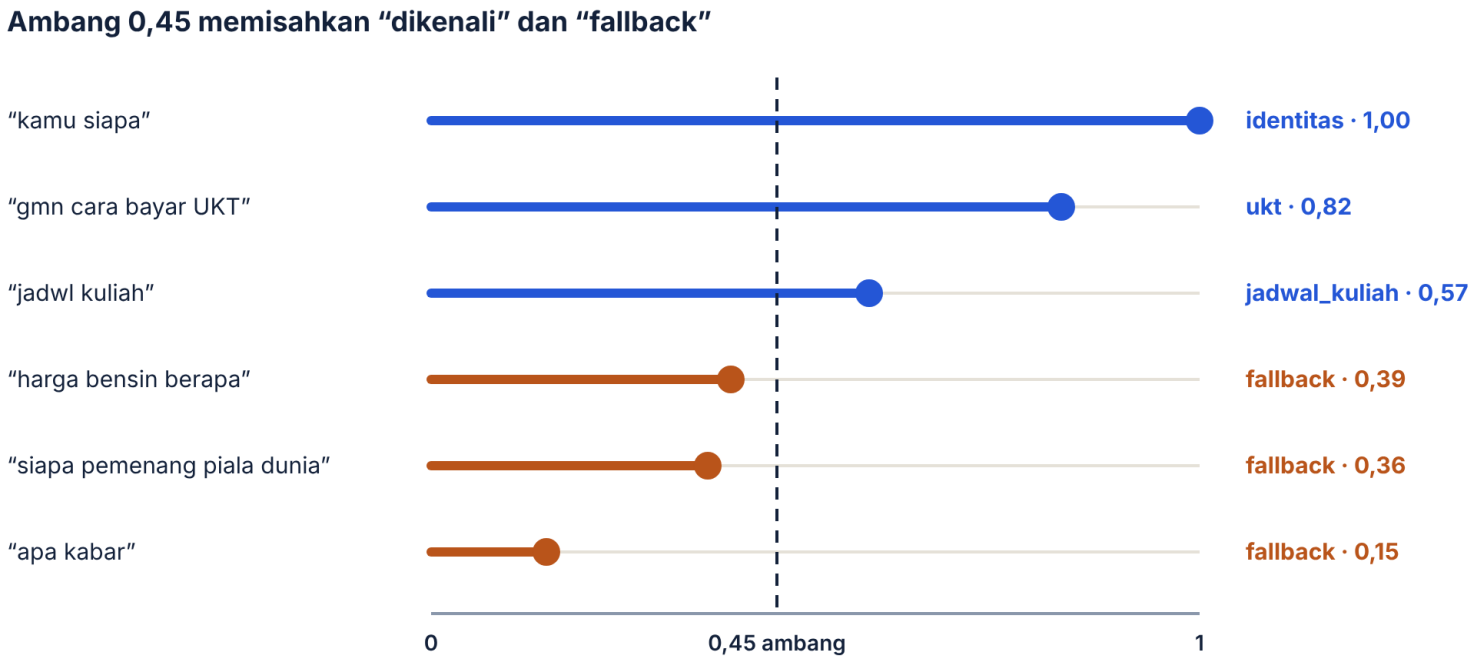
Pesan “gmn cara bayar UKT” melawan lima pola

Token pesan: `[bagaimana, cara, bayar, ukt]`

Pola di JSON	Token pola	Jaccard	Fuzzy	Cakupan	Skor
cara bayar ukt	[cara, bayar, ukt]	0,75	0,74	1,00	0,82
batas bayar ukt	[batas, bayar, ukt]	0,40	0,67	0,67	0,56
ukt	[ukt]	0,25	0,22	1,00	0,47
cara isi krs	[cara, isi, krs]	0,17	0,33	0,33	0,27
pinjam buku	[pinjam, buku]	0,00	0,34	0,00	0,10

Pola teratas: 3 kata sama ÷ 4 kata gabungan = Jaccard 0,75; ketiga kata pola muncul = cakupan 1,00.
0,4 × 0,75 + 0,3 × 0,74 + 0,3 × 1,00 = 0,82 → intent ukt menang.

Ambang 0,45 memisahkan paham dan tidak paham



Ambang terlalu rendah

Pertanyaan di luar tema ikut dijawab dengan intent yang salah.

Ambang terlalu tinggi

Pertanyaan yang sebenarnya bisa dijawab justru ditolak.

Akurasi tertinggi ada di ambang 0,40–0,45



93,1%

akurasi tertinggi, di ambang 0,40 dan 0,45

58,6%

akurasi di ambang 0,70: bot terlalu sering menolak

Sumber: python evaluate.py --tema kampus --sweep

KUNCI MEMAHAMI CHATBOT INI

Bot tidak “mengerti” bahasa. Ia mengukur seberapa mirip kata-kata Anda dengan pola yang sudah ditulis.

Karena itu kualitas chatbot ditentukan oleh kualitas pola di file JSON.

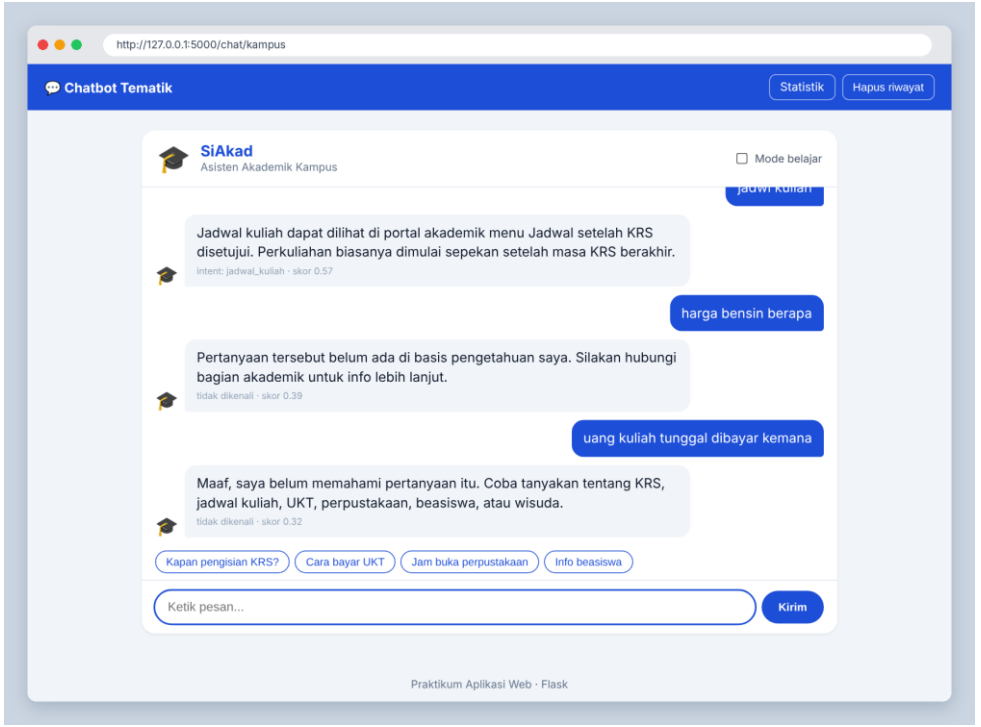
MENJAWAB

Jawaban dibuat dinamis: dipilih acak dan diisi placeholder

chatbot/engine.py

```
def reply(self, pesan):
    tag, skor = self.classify(pesan)
    if tag is None:
        teks = random.choice(self.theme["fallback"])
    else:
        teks = random.choice(self._responses[tag])
    return {"reply": self._isi_placeholder(teks),
            "intent": tag, "score": skor}
```

Placeholder	Diganti dengan
{waktu}	Jam server, misal 10:12
{tanggal}	Tanggal, misal 05-11-2026
{nama_bot}	Nama bot dari file tema



Pertanyaan di luar tema mendapat fallback dengan skor di bawah 0,45

TAHAP 3

Mesin chatbot bisa dicoba di terminal, tanpa web




```
python bandingkan.py
python cli.py --debug
python cli.py --versi 1
```

- Tidak perlu server: cukup Python
- Tiga kandidat teratas beserta pola dan skornya
- Ketik keluar untuk berhenti
- Aplikasi final punya cli.py yang sama untuk semua tema

05

API dan Antarmuka

Membungkus mesin chatbot menjadi REST API, lalu membuat halaman chat yang tidak perlu dimuat ulang.

REST API: program berbicara dengan program lewat JSON

Endpoint

Alamat + metode, misalnya POST /api/chat.

JSON masuk

Body request: {"message": "..."}

JSON keluar

Response: {"reply", "intent", "score"}

Stateless

Setiap request berdiri sendiri; riwayat disimpan terpisah.

Jenis route	Mengembalikan	Untuk	Contoh
Halaman	HTML	Manusia (browser)	GET /chat/kampus
API	JSON	Program (JavaScript, aplikasi lain)	POST /api/chat

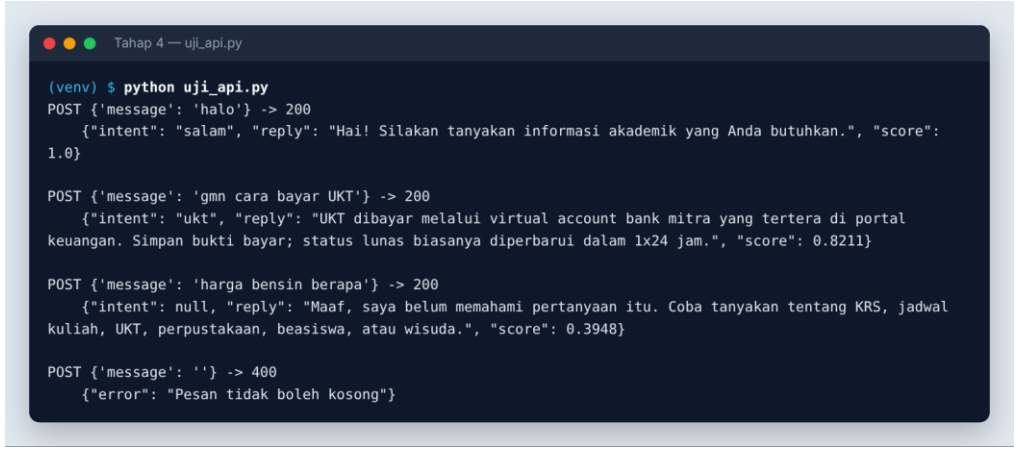
TAHAP 4

Endpoint POST /api/chat dengan validasi input

tahapan/tahap-04-api-json/app.py (inti)

```
def galat(pesan):
    return jsonify({"error": pesan}), 400

@app.post("/api/chat")
def api_chat():
    data = request.get_json(silent=True)
    if data is None:
        return galat("Body harus berformat JSON")
    message = str(data.get("message", "")).strip()
    if not message:
        return galat("Pesan tidak boleh kosong")
    if len(message) > 500:
        return galat("Pesan maksimal 500 karakter")
    return jsonify(bot.reply(message))
```



- Validasi di awal: mesin tidak pernah menerima input buruk
- Kode 400 + pesan galat yang jelas untuk JavaScript

uji_api.py: tiga request dijawab 200, pesan kosong dijawab 400

fetch() mengirim pesan tanpa memuat ulang halaman

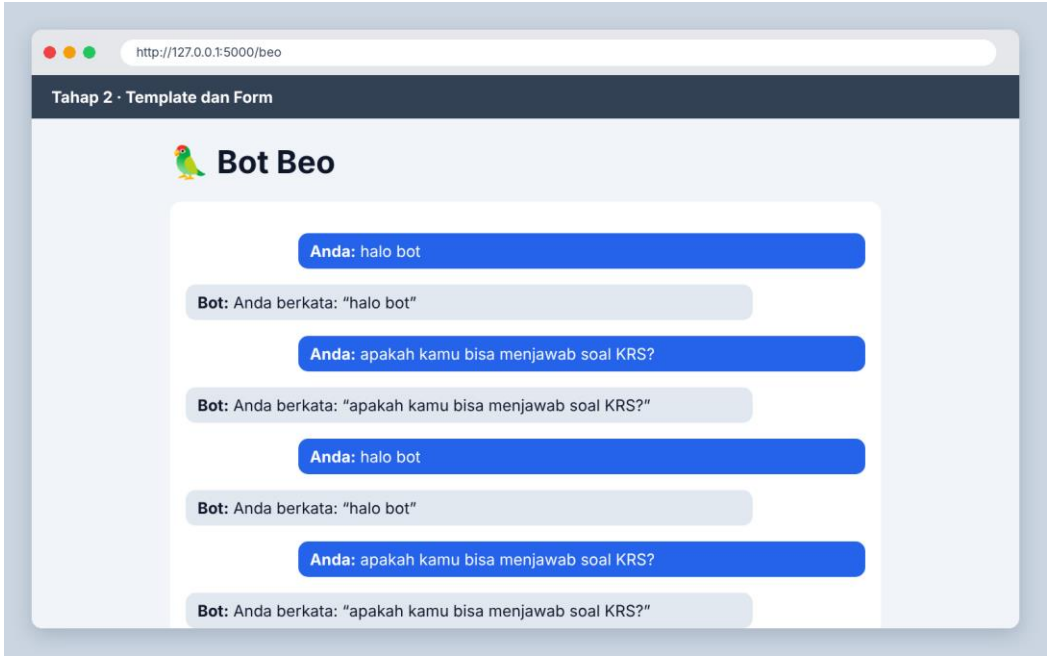
tahapan/tahap-05-antarmuka-chat/static/chat.js

```
async function sendMessage(text) {
  addMessage("user", text);
  try {
    const res = await fetch("/api/chat", {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({message: text}),
    });
    const data = await res.json();
    if (!res.ok) return addMessage("bot", data.error);
    addMessage("bot", data.reply);
  } catch (err) {
    addMessage("bot", "Gagal terhubung ke server.");
  }
}
```

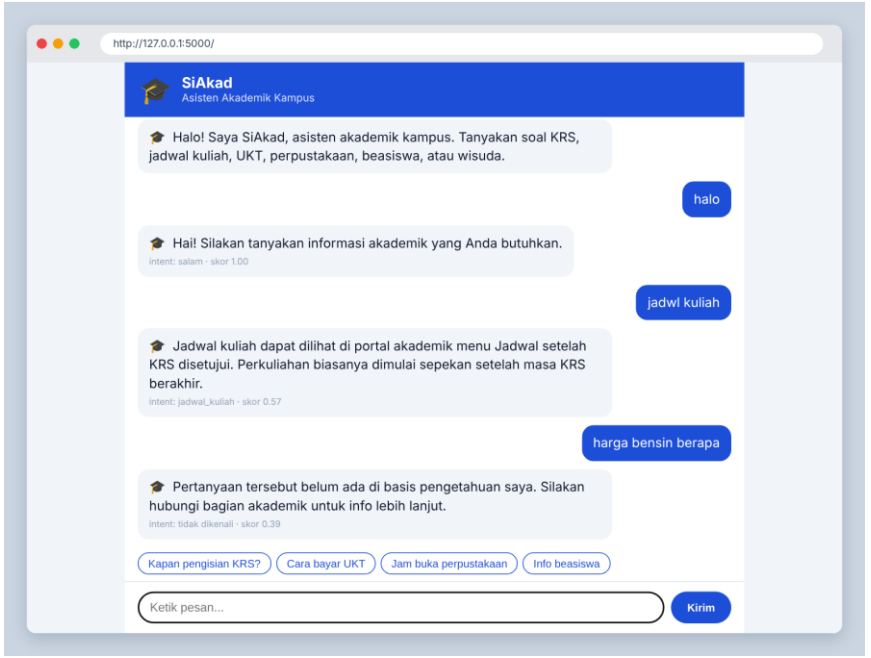
1. await fetch(): kirim POST berisi JSON
2. await res.json(): baca jawaban
3. res.ok: cek kode status
4. catch: tangani server mati

TAHAP 5

Form biasa memuat ulang halaman, fetch() tidak



Tahap 2 · form POST: seluruh halaman dimuat ulang

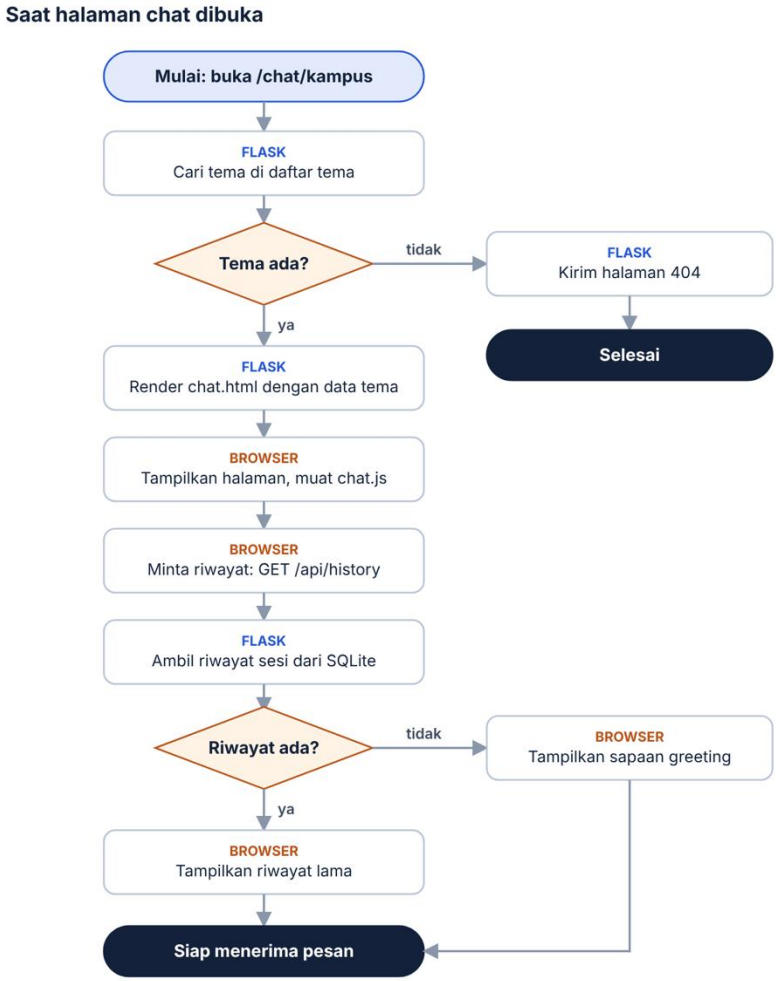


Tahap 5 · fetch(): hanya gelembung baru yang ditambahkan

	Form POST (Tahap 2)	fetch() + API (Tahap 5)
Yang dikirim server	Seluruh halaman HTML	Hanya JSON jawaban
Pengalaman pengguna	Layar berkedip, gulir hilang	Mulus, seperti aplikasi chat

FLOWCHART · 1

Saat halaman chat dibuka



Keputusan 1 · Tema ada?

Flask memeriksa daftar tema. Tidak ada → halaman 404 dan selesai.

Keputusan 2 · Riwayat ada?

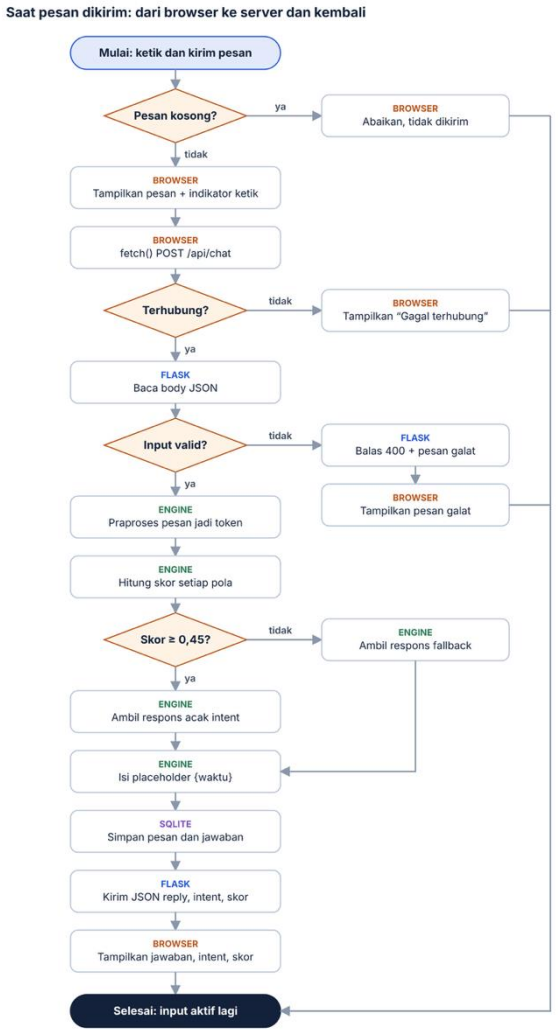
chat.js meminta riwayat sesi. Kosong → tampilkan sapaan dari file tema.

Siapa mengerjakan apa

FLASK di server, BROWSER di komputer pengguna. Keduanya berbicara lewat HTTP.

FLOWCHART · 2

Saat pesan dikirim: empat keputusan



Pesan kosong?

Dicek di browser; tidak dikirim ke server.

Terhubung?

Gagal → blok catch menampilkan “Gagal terhubung”.

Input valid?

Dicek di Flask; tidak valid → kode 400.

Skor ≥ 0,45?

Dicek di engine; tidak → respons fallback.

Gabungan dua sisi: kotak BROWSER di chat.js, FLASK di app.py, ENGINE di engine.py, SQLITE di database.py.

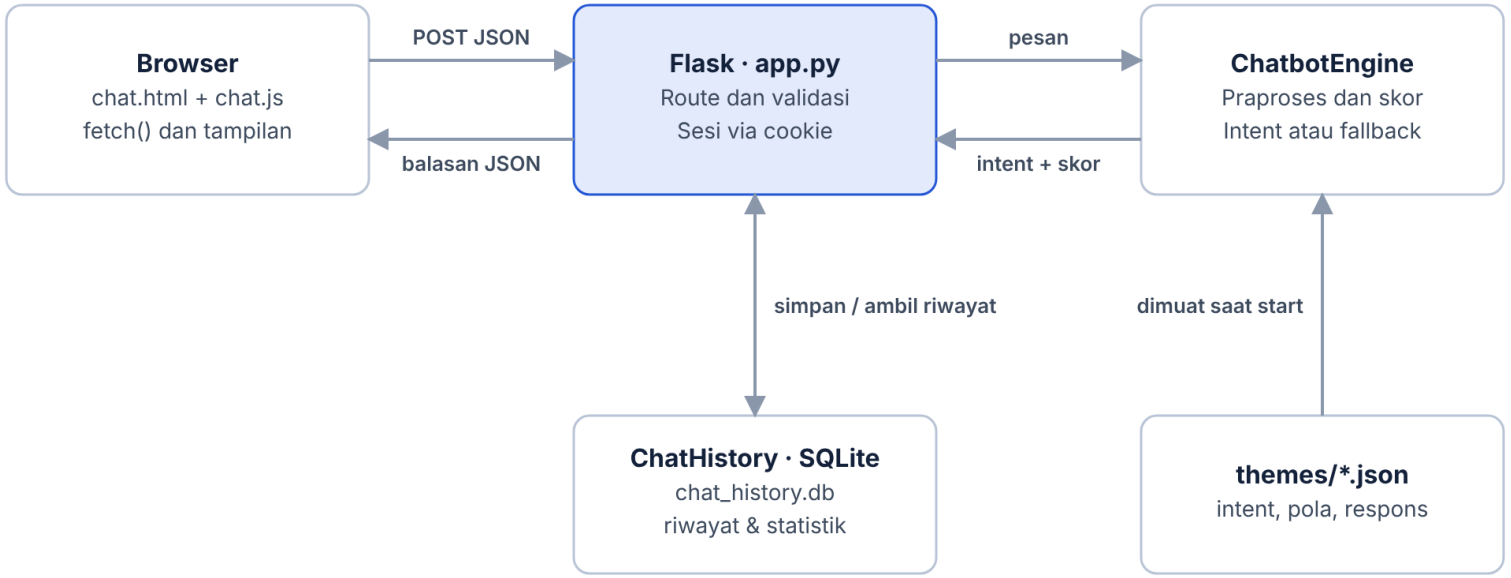
06

Aplikasi Final

Banyak tema, sesi pengguna, riwayat SQLite, halaman admin, dan pengaman dasar.

Lima komponen, satu alur

Arsitektur aplikasi final: lima komponen, satu alur



- **Browser:** menampilkan, mengirim, menerima
- **Flask:** route, validasi, sesi
- **Engine:** praproses, skor, keputusan
- **Tema JSON:** seluruh pengetahuan bot
- **SQLite:** riwayat dan statistik

ROUTE

Tiga halaman HTML dan enam endpoint JSON

Metode	URL	Mengembalikan	Keterangan
GET	/	HTML	Halaman pilih tema
GET	/chat/<tema>	HTML	Halaman chat
GET	/admin/<tema>	HTML	Statistik dan pertanyaan tak dikenali
GET	/api/themes	JSON	Daftar tema
POST	/api/chat	JSON	Kirim pesan, terima jawaban
POST	/api/explain	JSON	Tiga kandidat intent teratas
GET	/api/history/<tema>	JSON	Riwayat sesi ini
DELETE	/api/history/<tema>	JSON	Hapus riwayat sesi ini
GET	/api/stats/<tema>	JSON	Jumlah pertanyaan per intent

Sesi dan cookie: membedakan pengunjung satu dengan lainnya

```
def get_session_id():
    if "sid" not in session:
        session["sid"] = uuid.uuid4().hex
    return session["sid"]
```

Mengapa ditandatangani?

Isi cookie bisa dibaca pengguna, tetapi tidak bisa diubah tanpa SECRET_KEY.

1 · Kunjungan pertama

Flask membuat ID acak, misal 9f3c…, dan menitipkannya di cookie.

2 · Request berikutnya

Browser otomatis mengirim cookie; Flask tahu ini pengunjung yang sama.

3 · Riwayat terpisah

Riwayat di SQLite disaring per session_id dan tema.

SQLITE

Riwayat disimpan di satu tabel SQLite

Kolom	Tipe	Isi
id	INTEGER	Kunci utama, otomatis
session_id	TEXT	ID pengunjung dari cookie
theme	TEXT	kampus, kuliner, wisata
sender	TEXT	user atau bot
text	TEXT	Isi pesan
intent	TEXT	Intent yang dikenali (kosong = tidak dikenali)
score	REAL	Skor tertinggi
created_at	TIMESTAMP	Waktu lokal, otomatis

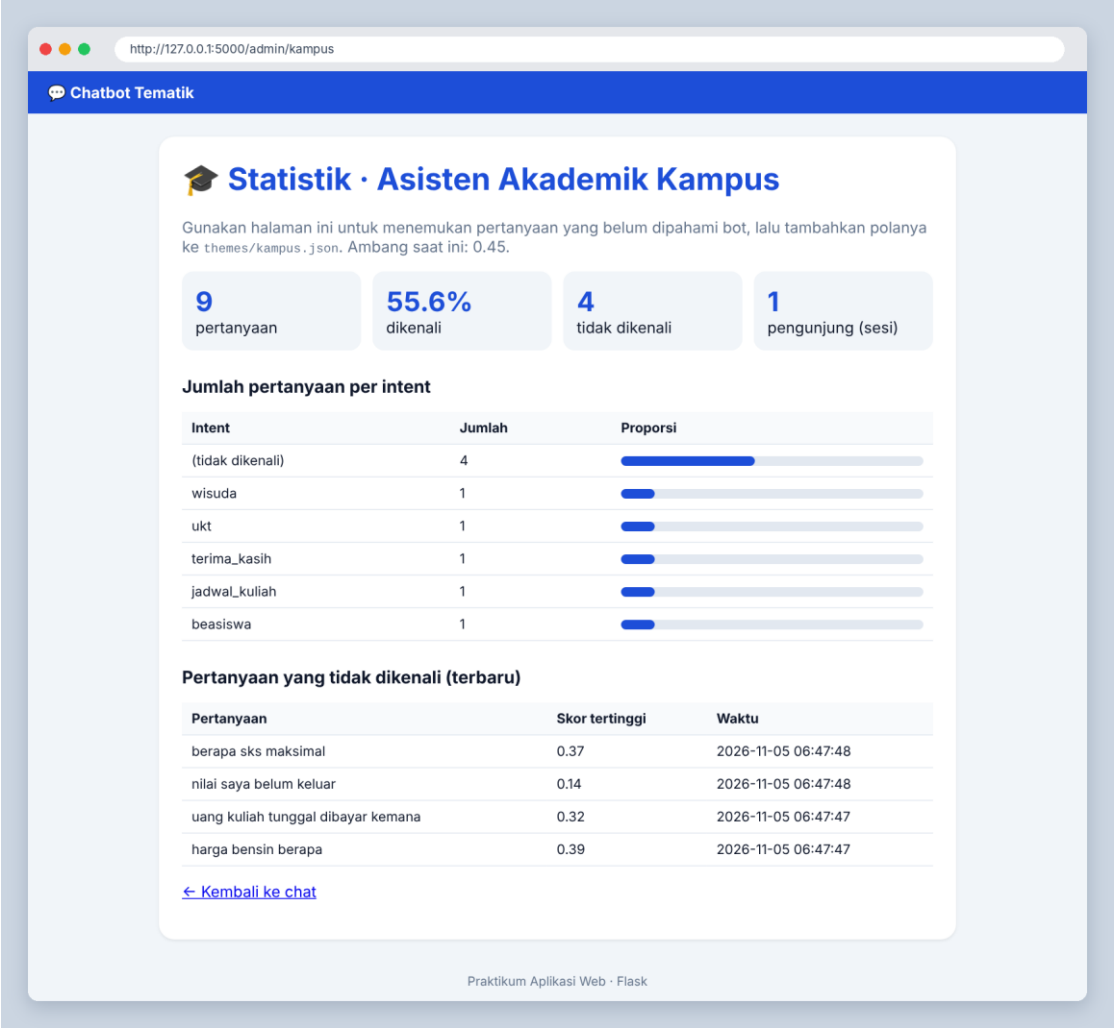
```
-- simpan (parameter ? mencegah SQL injection)
INSERT INTO messages (session_id, theme, sender,
  text, intent, score) VALUES (?, ?, ?, ?, ?, ?)

-- statistik per intent
SELECT COALESCE(intent, '(tidak dikenali)'),
       COUNT(*) FROM messages
WHERE theme = ? AND sender = 'user'
GROUP BY intent ORDER BY 2 DESC
```

with conn: otomatis commit bila berhasil dan rollback bila galat.

HALAMAN ADMIN

Halaman admin menunjukkan apa yang belum dipahami bot



Ringkasan angka

Jumlah pertanyaan, persen dikenali, jumlah sesi.

Per intent

Topik apa yang paling sering ditanyakan.

Tidak dikenali

Kalimat aslinya → salin sebagai pola baru di JSON.

Application factory memudahkan pengujian

```
def create_app(config=None):
    app = Flask(__name__)
    app.config.update(
        SECRET_KEY=os.environ.get("SECRET_KEY", "..."),
        THRESHOLD=float(os.environ.get("THRESHOLD", "0.45")),
        ADMIN_KEY=os.environ.get("ADMIN_KEY", ""))
    if config:
        app.config.update(config)
    themes = load_themes(app.config["THEMES_DIR"])
    engines = {tid: ChatbotEngine(t) for tid, t in themes.items()}
    history = ChatHistory(app.config["DATABASE"])
     # ... route didefinisikan di sini ...
    return app
```

Di tes

```
app = create_app({"TESTING": True,
  "DATABASE": str(tmp_path / "t.db")})
```

Variabel lingkungan	Bawaan
SECRET_KEY	wajib diganti
THRESHOLD	0.45
DATABASE	chat_history.db
ADMIN_KEY	kosong

KEAMANAN

Enam pengaman dasar aplikasi web



Cegah XSS

Tampilkan pesan dengan textContent, bukan innerHTML.



Cegah SQL injection

Nilai pengguna lewat parameter ?, tidak digabung ke SQL.



Validasi input

Tema harus ada, pesan tidak kosong dan maks. 500 karakter.



Rahasiakan SECRET_KEY

Ambil dari variabel lingkungan, jangan diunggah.



Lindungi admin

Isi ADMIN_KEY agar /admin butuh ?key=...



Matikan debug

debug=True hanya di komputer sendiri.

07

Pengujian dan Evaluasi

Pengujian memastikan kode benar; evaluasi mengukur seberapa baik bot memahami pengguna.

PENGUJIAN

34 uji otomatis menjaga aplikasi tetap benar

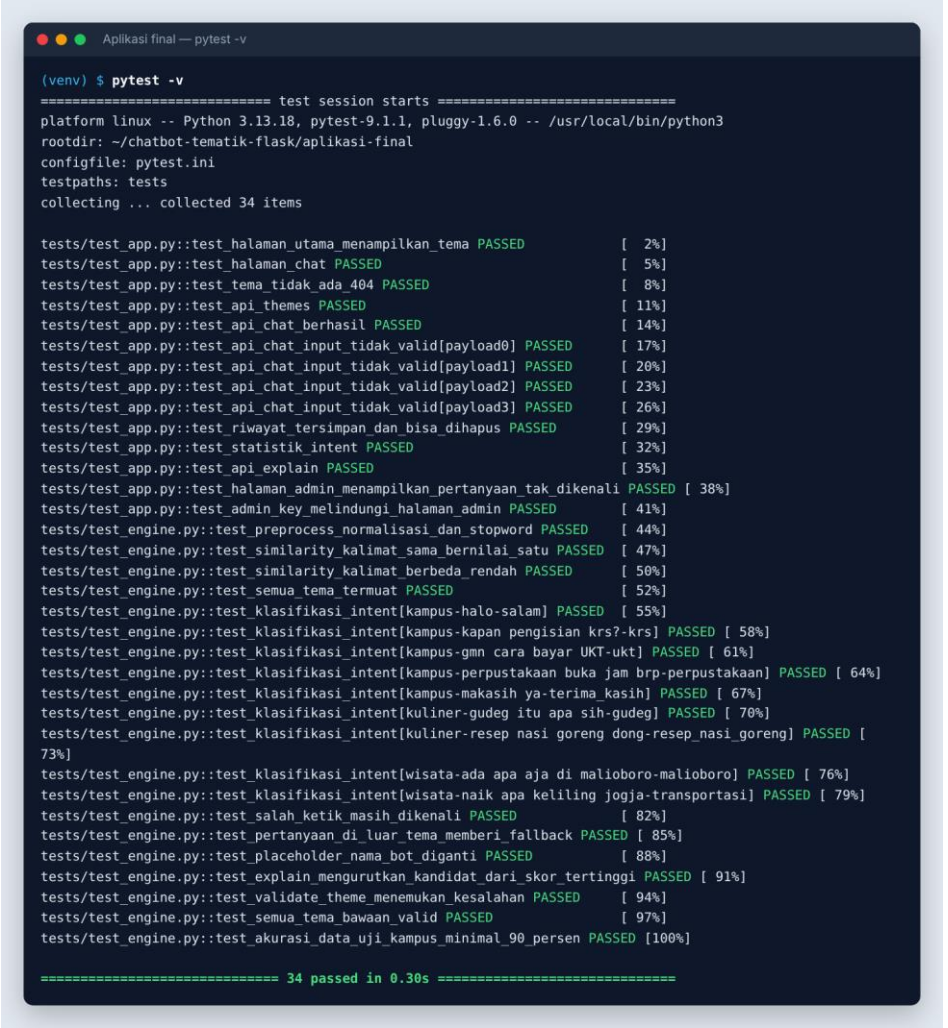
Uji unit · 20 uji (test_engine.py)

Praproses, skor, klasifikasi intent, validasi tema, akurasi minimal 90%.

```
def test_salah_ketik_masih_dikenali():
    tag, _ = engine.classify("jadwl kuliah")
    assert tag == "jadwal_kuliah"
```

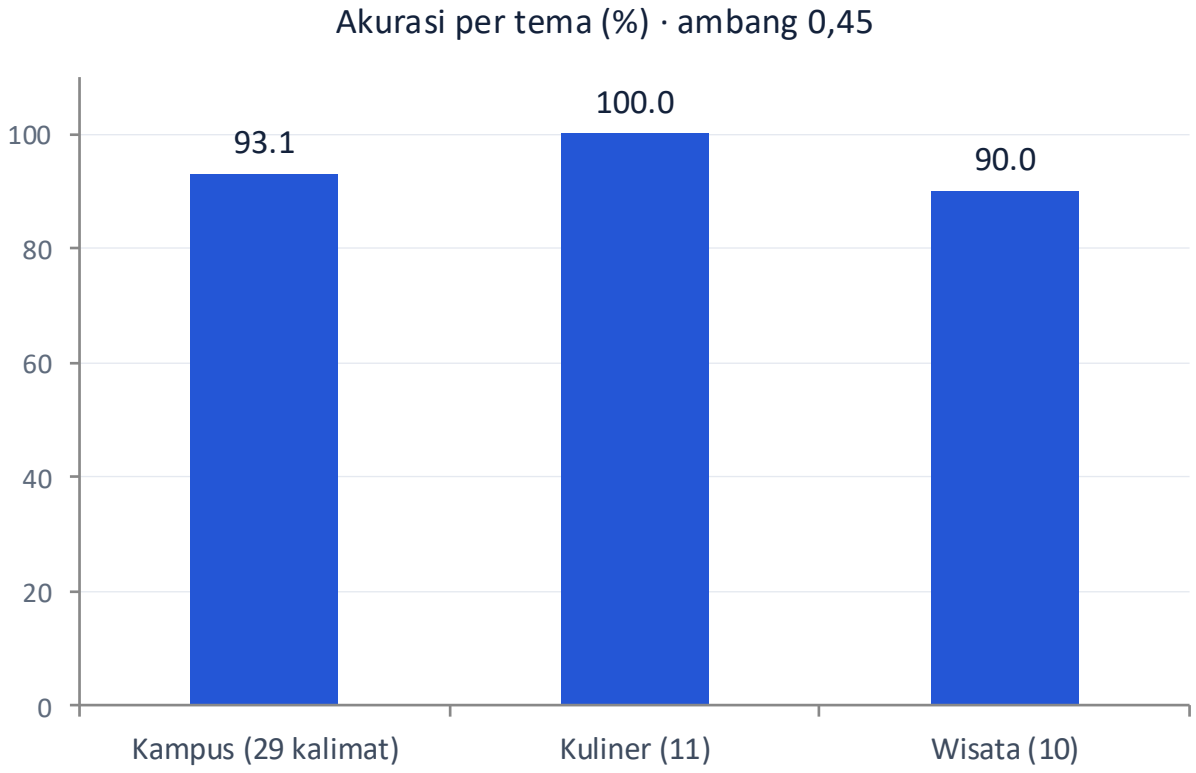
Uji integrasi · 14 uji (test_app.py)

Route dipanggil lewat Flask test client dengan database sementara: kode status, riwayat, admin, kunci admin.



EVALUASI

Evaluasi mengukur akurasi dengan kalimat baru



Sumber: python evaluate.py --tema <tema>

evaluasi/kampus.csv

```
kalimat,intent
perpus buka jam berapa,perpustakaan
ukt bayarnya sampai kapan,ukt
siapa presiden indonesia,fallback
```

Akurasi = kalimat yang ditebak benar ÷ semua kalimat uji

Tema kampus: 27 benar dari 29 = 93,1%. Dua yang salah sama-sama memakai sinonim.

- Data uji berbeda dari pola di JSON
- Memuat pertanyaan di luar tema yang harus ditolak
- python evaluate.py --tema kampus --sweep membandingkan ambang

KETERBATASAN

Batas pendekatan berbasis aturan dan jalan keluarnya

Masalah	Contoh	Jalan keluar
Sinonim	“uang kuliah tunggal” ≠ “ukt”	Tambah pola; kamus sinonim di NORMALISASI
Imbuhan	“pembayaran” ≠ “bayar”	Stemming (misal pustaka Sastrawi)
Kata umum dominan	“berapa” muncul di banyak pola	Bobot kata TF-IDF
Makna, bukan kata	“tempat baca buku” = perpustakaan	Embedding kalimat atau model ML
Pertanyaan bebas	Di luar daftar intent	LLM sebagai fallback yang dibatasi tema
Konteks percakapan	“kalau yang S2?” setelah tanya UKT	Simpan intent terakhir di sesi

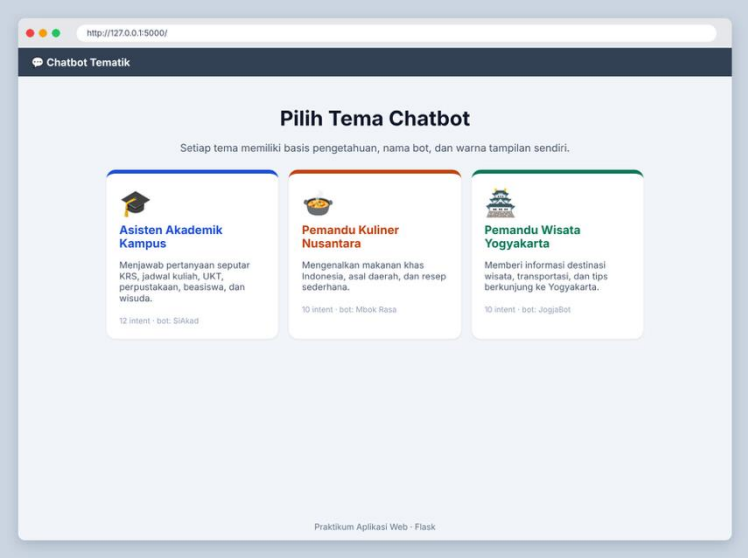
08

Hasil, Deployment, dan Tugas

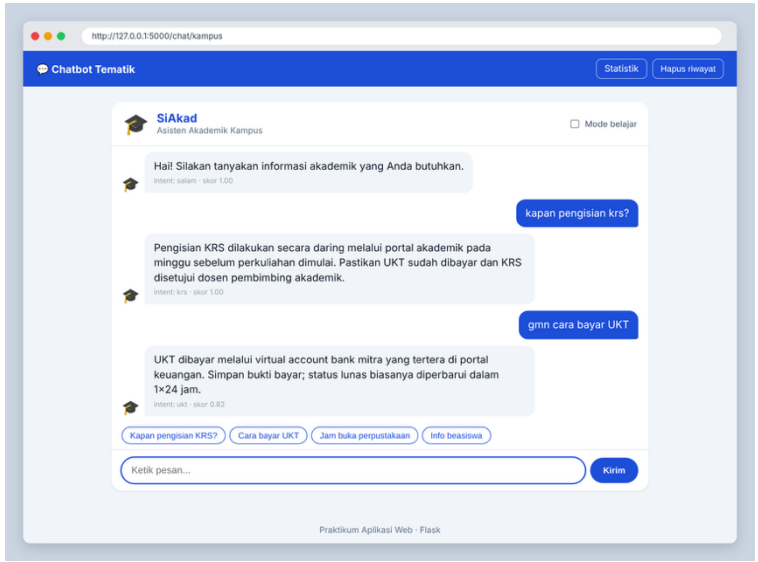
Seperti inilah aplikasi yang jadi, cara membawanya ke server, dan apa yang perlu dikerjakan.

HASIL

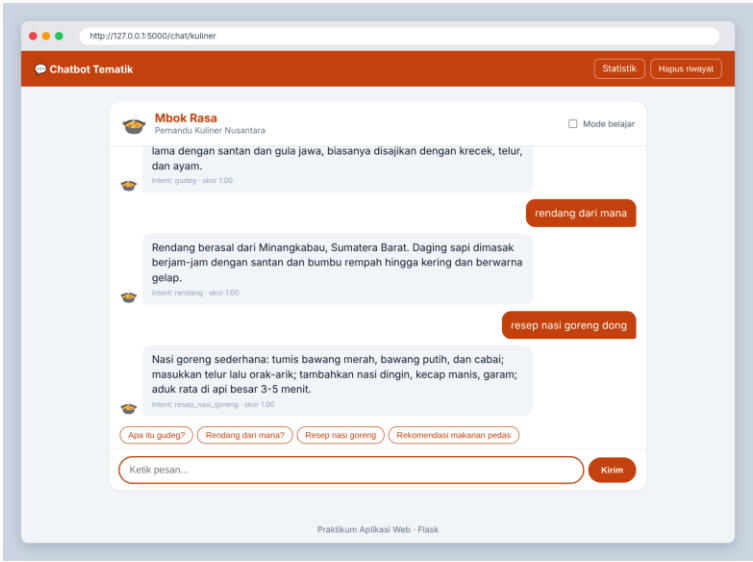
Satu kode, tiga tema



Halaman utama · kartu dibuat otomatis dari folder themes/



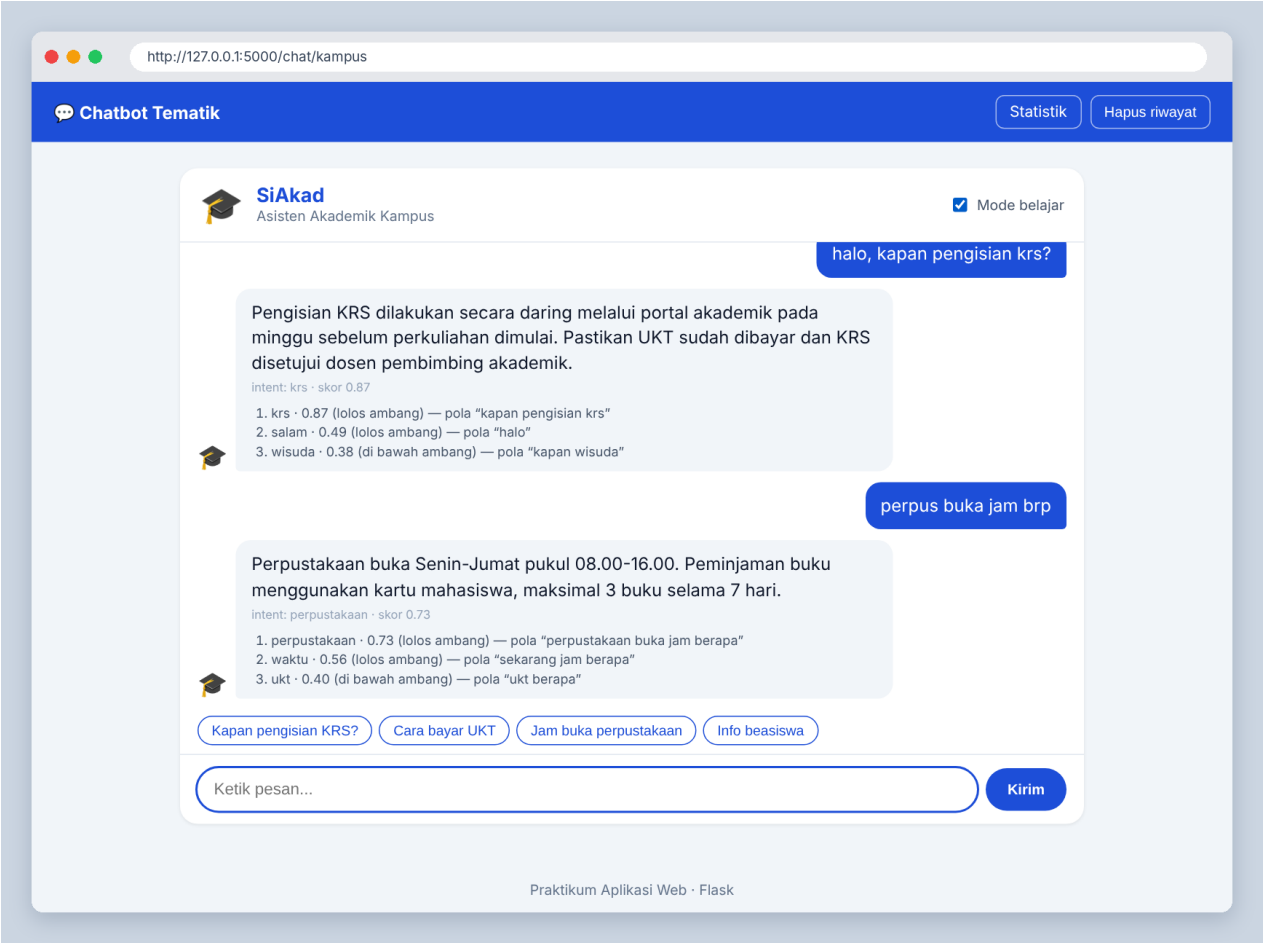
Kampus · intent dan skor di bawah jawaban



Kuliner · warna dan persona berbeda

Menambah tema baru = menyalin template-tema.json ke themes/, mengisinya, lalu menjalankan ulang app.py.

Mode belajar dan tampilan ponsel





"halo, kapan pengisian krs?": krs 0,87 mengalahkan salam 0,49

DEPLOYMENT

Dari komputer sendiri ke server sungguhan

Pengembangan (sekarang)

python app.py · debug=True · hanya bisa dibuka dari komputer sendiri (127.0.0.1).

Produksi

Server WSGI (gunicorn/waitress) · debug mati · HTTPS · domain sendiri.

```
pip install waitress
waitress-serve --port=8000 app:app      # Windows/Linux

pip install gunicorn
gunicorn -w 2 -b 0.0.0.0:8000 app:app   # Linux
```

- Isi SECRET_KEY dan ADMIN_KEY lewat variabel lingkungan
- Pastikan debug tidak aktif
- Cadangkan chat_history.db secara berkala
- Jalankan pytest sebelum setiap pembaruan

TUGAS

Empat tugas wajib

1 · Tema baru

themes/<tema>.json: ≥ 10 intent, ≥ 5 pola dan 2 respons per intent, 4 saran.

2 · Uji tema

≥ 8 kasus uji di tests/test_engine.py, semua lolos.

3 · Normalisasi

≥ 10 kata tidak baku yang relevan di NORMALISASI.

4 · Evaluasi

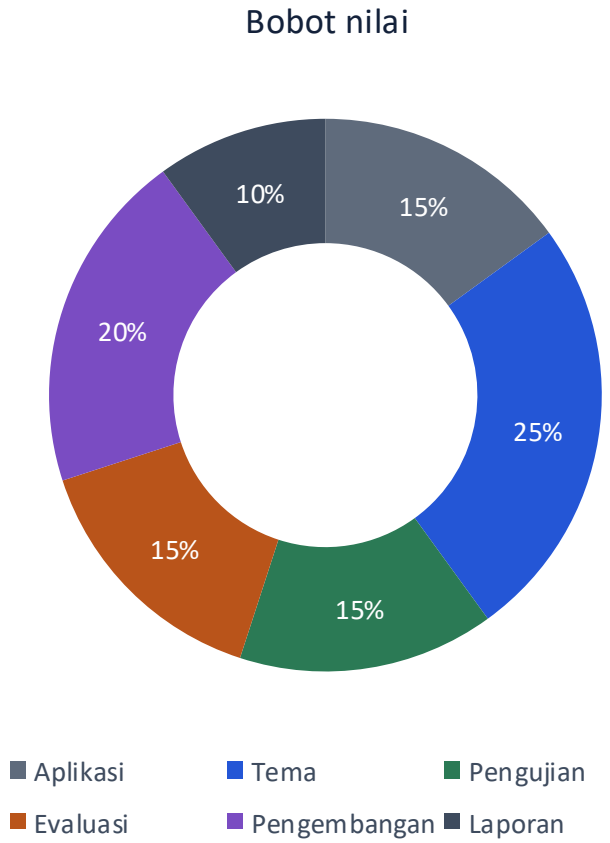
evaluasi/<tema>.csv ≥ 20 kalimat + uji coba 3 teman; laporkan akurasi dan perbaikan.

Dikumpulkan: ZIP proyek (tanpa venv/ dan *.db) + laporan PDF maksimal 5 halaman.

PENILAIAN

Rubrik penilaian

Aspek	Bobot	Kriteria nilai penuh
Aplikasi berjalan	15%	python app.py tanpa galat, semua halaman bisa dibuka
Basis pengetahuan tema	25%	≥ 10 intent, pola bervariasi dan realistis, jawaban akurat
Pengujian	15%	≥ 8 uji tema baru, seluruh uji lolos
Evaluasi dan perbaikan	15%	Akurasi dilaporkan sebelum dan sesudah perbaikan
Tugas pengembangan	20%	Satu fitur lanjutan berfungsi, kode rapi dan berkomentar
Laporan	10%	Lengkap, jelas, pertanyaan pemahaman terjawab



PENGEMBANGAN

Pilih satu arah tugas pengembangan



Grafik di admin

Tampilkan statistik intent sebagai grafik (Chart.js).



Intent berkonteks

Kunci "next" agar bot bertanya lanjutan.



Pencocokan TF-IDF

TfidfVectorizer + cosine similarity, bandingkan akurasinya.



LLM sebagai fallback

Panggil API model bahasa yang dibatasi tema; kunci di variabel lingkungan.



Mode gelap

Tombol terang/gelap dan navigasi keyboard.



Ide sendiri

Usulkan ke dosen beserta rencana pengujiannya.

RINGKASAN

Tujuh hal yang perlu diingat

Chatbot tematik	Domain sempit = jawaban pasti benar dan akurasi terukur.
HTTP	Browser mengirim request, Flask membalas HTML atau JSON.
Flask	Route memetakan URL ke fungsi; Jinja2 mengisi template.
Tema JSON	Seluruh pengetahuan bot terpisah dari kode.
Mesin chatbot	Praproses → skor 0,4 J + 0,3 F + 0,3 C → ambang 0,45.
API dan fetch()	POST /api/chat; jawaban tampil tanpa reload.
Uji dan evaluasi	34 tes menjaga kode; data uji mengukur akurasi (kampus 93,1%).

PENUTUP

Selamat mencoba!

Kode: chatbot-tematik-flask.zip · Panduan menjalankan: CARA-MENJALANKAN.md · Langkah rinci: modul tutorial. Ada pertanyaan?